EE520: Deep Learning & Optimization Theory

Lecture Notes: Optimization, Backpropagation, and Transformers

Section 1: Gradient Descent Optimization

Gradient Descent iteratively updates model parameters theta in the direction opposite to the gradient of the loss function: theta = theta - eta * grad(L(theta)), where eta is the learning rate.

- **Batch Gradient Descent:** Computes gradients over the entire dataset. Deterministic updates but computationally prohibitive for large datasets.
- **Stochastic Gradient Descent (SGD):** Updates weights after computing loss on a single random training sample. High noise allows escaping local minima.
- **Mini-Batch Gradient Descent:** Uses batches of 32-512 examples, balancing vectorization GPU hardware efficiency with update stability.
- **Adam Optimizer:** Adaptive Moment Estimation tracks both first raw moment (momentum, beta1=0.9) and second uncentered moment (RMSProp variance, beta2=0.999).

Section 2: Transformer Architecture & Attention Mechanism

Vaswani et al. (2017) introduced 'Attention Is All You Need', replacing recurrence and convolutions with self-attention. The core mathematical formulation of Scaled Dot-Product Attention is:

Attention(Q, K, V) = softmax((Q * K^T) / sqrt(d_k)) * V
Where Q represents Queries, K represents Keys, V represents Values, and d_k is the dimension of keys. The scaling factor 1 / sqrt(d_k) prevents dot products from growing excessively large, which would push the softmax function into regions with vanishingly small gradients.

Multi-Head Attention projects Q, K, and V into h distinct lower-dimensional representation subspaces, allowing the model to jointly attend to information from different positions and representation subspaces simultaneously.

Section 3: Retrieval-Augmented Generation (RAG) Foundations

Retrieval-Augmented Generation (Lewis et al., 2020) addresses LLM limitations including hallucination, lack of domain knowledge, and static knowledge cutoff dates. A standard RAG pipeline operates in three phases:

1. **Indexing:** Documents are cleaned, chunked into semantically coherent segments, and embedded into high-dimensional vector representations.
2. **Retrieval:** When a user asks a query, the system retrieves top-k relevant chunks using vector cosine similarity, dense embeddings, or hybrid BM25 + dense search.
3. **Generation:** The retrieved context chunks are inserted into a structured prompt alongside the user prompt, instructing the LLM to generate grounded answers with source citations.

Evaluation Metrics for RAG:

• **Faithfulness / Groundedness:** Whether the generated answer can be strictly inferred from the retrieved context.
• **Answer Relevance:** Whether the answer directly addresses the user query.
• **Context Precision & Recall:** How relevant and complete the retrieved context chunks are relative to the ground truth.